Name – Siddhesh Wagh

Registration No. – 25BCE10169

Topic – Game Recommender System

Course -- Fundamentals of AI and ML

GAME RECOMMENDER SYSTEM

---

1. Title of the Project

Steam Game Recommender System

---

2. Abstract

The Steam Game Recommender System is a Python-based application designed to suggest games based on user preferences. It uses a hybrid approach combining genre-based filtering, popularity ranking, and similarity analysis. The system supports multi-genre input and provides dynamic, non-repetitive recommendations. It enhances user experience by helping players discover relevant games efficiently.

---

3. Introduction

With the rapid growth of digital gaming platforms like Steam, users are often overwhelmed by the large number of available games. Selecting the right game becomes difficult without proper guidance. This project aims to simplify game discovery by recommending games based on user-selected genres and previously liked games.

---

4. Objectives

- Recommend games based on multiple genres
- Suggest games similar to a user’s favorite game
- Rank games using popularity metrics
- Avoid repetition in recommendations
- Provide a simple and interactive user interface

---

5. Technologies Used

- Python 3.x
- CSV Module
- Random Module

---

6. System Requirements

Hardware

- Minimum 4GB RAM
- Intel i3 or higher processor
- 1GB storage

Software

- Python 3.x
- IDE (VS Code / PyCharm / IDLE)

---

7. Methodology

1. Load dataset containing game details.
2. Extract game name, tags, and popularity metrics.
3. Detect genres using keyword matching.
4. Accept user input (genre or game name).
5. Apply recommendation logic:
    - Genre-based filtering
    - Similarity-based filtering
6. Rank games using popularity score.
7. Select top results with randomness for diversity.
8. Display recommendations to the user.

---

8. System Flow Diagram

START → Load Dataset → User Chooses Option →
(Genre Input / Game Input) →
Process Data → Rank Results →
Display Recommendations → END

---

9. Pseudocode

```
BEGIN
DISPLAY "STEAM GAME RECOMMENDER SYSTEM"

WHILE true
   DISPLAY menu options
   INPUT user choice
```

```
    IF choice = genre
        INPUT genres
        FILTER games
        CALCULATE scores
        DISPLAY results

    ELSE IF choice = similar
        INPUT game name
        FIND similar games
        DISPLAY results

    ELSE IF exit
        BREAK

END WHILE

END
```

---

10. Source Code

```
import csv

import random


# Define genres

GENRES = {

    "fps": ["fps", "shooter", "gun", "first person"],

    "horror": ["horror", "scary", "fear", "evil", "ghost"],

    "rpg": ["rpg", "role", "fantasy", "souls"],

    "open_world": ["open", "world", "sandbox"],

    "survival": ["survival", "craft", "zombie"],

    "multiplayer": ["multiplayer", "online", "co-op"],

    "racing": ["racing", "cars", "driving"],

    "sports": ["sports", "football", "basketball"],

    "strategy": ["strategy", "rts", "tactics"],

    "simulation": ["simulation", "simulator"],

    "adventure": ["adventure", "story"],

    "action": ["action", "combat", "fight"],
```

```
    "stealth": ["stealth", "sneak", "assassin"]
}

games = []
seen_games = set()  # Prevent repetition

# Popularity score
def get_popularity_score(row):
    try:
        positive = int(row.get("positive", 0))
        negative = int(row.get("negative", 0))
        owners = row.get("owners", "0")

        if "-" in owners:
            owners = int(owners.split("-")[0])
        else:
            owners = int(owners)

        return positive - negative + (owners // 1000)
    except:
        return 0

# Load dataset
with open("steam_games_large.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
        name = row["name"]
        tags = row["tags"].lower()
```

```
        if name and tags:
            games.append({
                "name": name,
                "tags": tags,
                "score": get_popularity_score(row)
            })

print(f"✅ Loaded {len(games)} games\n")

# Detect genres
def detect_game_genres(game):
    detected = []

    for genre, keywords in GENRES.items():
        for word in keywords:
            if word in game["tags"]:
                detected.append(genre)
                break

    return detected

# Genre-based recommender (NO REPEAT + VARIETY)
def recommend_by_genres(user_genres, top_n=7):
    results = []

    for game in games:
        detected = detect_game_genres(game)
        match_count = len(set(user_genres) & set(detected))
```

```
        if match_count > 0:
            final_score = (match_count * 10) + game["score"]
            results.append((final_score, game))

    results.sort(key=lambda x: x[0], reverse=True)

    # Top pool for variety
    top_pool = [game for _, game in results[:50]]

    # Remove seen games
    filtered = [g for g in top_pool if g["name"] not in seen_games]

    if len(filtered) < top_n:
        seen_games.clear()
        filtered = top_pool

    selected = random.sample(filtered, min(top_n, len(filtered)))

    for g in selected:
        seen_games.add(g["name"])

    return selected

# Similar games recommender (NO REPEAT + VARIETY)
def recommend_similar_games(game_name, top_n=7):
    target_game = None

    for game in games:
```

```
    if game_name.lower() in game["name"].lower():
        target_game = game
        break

if not target_game:
    return []

target_tags = set(target_game["tags"].split(","))
similarities = []

for game in games:
    if game == target_game:
        continue

    game_tags = set(game["tags"].split(","))

    intersection = len(target_tags & game_tags)
    union = len(target_tags | game_tags)

    if union == 0:
        continue

    similarity_score = intersection / union
    final_score = (similarity_score * 100) + (game["score"] / 1000)

    similarities.append((final_score, game))

similarities.sort(key=lambda x: x[0], reverse=True)
```

```
    # Top pool
    top_pool = [game for _, game in similarities[:50]]

    filtered = [g for g in top_pool if g["name"] not in seen_games]

    if len(filtered) < top_n:
        seen_games.clear()
        filtered = top_pool

    selected = random.sample(filtered, min(top_n, len(filtered)))

    for g in selected:
        seen_games.add(g["name"])

    return selected

# MAIN UI
def main():
    print("ADVANCED Steam Recommender System\n")

    while True:
        print("\nChoose what you want to do:\n")
        print("1.Recommend by Genre")
        print("2. Recommend Games Like X")
        print("3. Exit")

        choice = input("\nEnter choice (1/2/3): ").strip()

        # GENRE
```

```
if choice == "1":
    print("\nAvailable genres:\n")
    for g in GENRES:
        print(f" {g}")

    user_input = input("\nEnter genres (comma-separated): ").lower()
    user_genres = [g.strip() for g in user_input.split(",")]

    valid_genres = [g for g in user_genres if g in GENRES]

    if not valid_genres:
        print("No valid genres entered")
        continue

    results = recommend_by_genres(valid_genres)

    if not results:
        print("No games found")
        continue

    print("\nRecommended Games:\n")
    for game in results:
        print(f"{game['name']}")

    print("\n" + "-"*40)

# SIMILAR
elif choice == "2":
    game_name = input("\nEnter a game you like: ")
```

```
            results = recommend_similar_games(game_name)

            if not results:
                print("Game not found or no similar games")
                continue

            print("\nSimilar Games:\n")
            for game in results:
                print(f"{game['name']}")

            print("\n" + "-"*40)

        # EXIT
        elif choice == "3":
            print("Exiting...")
            break

        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    main()
```

---

11. Core Analysis Explained

- Genre Matching: Identifies games matching user-selected genres
- Popularity Score: Ranks games using reviews and ownership
- Similarity Score: Measures similarity using tags
- Diversity System: Ensures varied recommendations

- Non-Repetition: Prevents duplicate suggestions
- Ranking Logic: Combines multiple scoring techniques

---

12. Genre Specific Recommendations

- FPS: shooting accuracy, reflex-based gameplay
- RPG: story depth, character progression
- Strategy: planning and resource management
- Open World: exploration and freedom
- Simulation: realistic gameplay experience

---

13. Sample Output

Input: action, open_world

Output:
🎯 GTA V
🎯 Red Dead Redemption 2
🎯 Cyberpunk 2077

---

14. Advantages

- Efficient game discovery
- Simple and user-friendly interface
- Dynamic recommendations
- Supports multiple genres
- Avoids repetition

---

15. Applications

- Gaming platforms
- Recommendation engines
- E-commerce systems
- Content suggestion systems

---

16. Limitations

- Not using advanced machine learning
- Depends on dataset quality
- Keyword-based genre detection

---

17. Future Scope

- Implement machine learning models
- Build web or mobile interface
- Integrate Steam API
- Add user personalization

---

18. Conclusion

The project successfully demonstrates a hybrid recommender system that suggests games based on user preferences. It provides a practical solution for game discovery and can be further enhanced using advanced AI techniques.

---

19. Bibliography

- Python Official Documentation
- Steam Dataset (Kaggle)
- Recommender System Concepts

END